Proposta de Rubrica de Correção das Sprints do ABP 1DSM – 2026-2

1. Regras gerais de avaliação

A rubrica completa totaliza 100 pontos. A equipe deve utilizar um único repositório nas três sprints. São consideradas evidências disponíveis até o fechamento: código, SQL, documentação, issues, pull requests e versões dos protótipos no Figma documentadas no repositório GitHub.

Para cada critério programado para a sprint, o avaliador atribuirá:

Resultado	Significado
Atendido	Há evidência verificável e o comportamento ou artefato corresponde ao critério.
Parcialmente atendido	Há implementação ou documentação, mas falta uma parte relevante, há inconsistência ou não foi possível verificar plenamente o funcionamento.
Não atendido	A evidência está ausente ou a implementação não satisfaz o critério.
Não avaliável	Existe um impedimento concreto para a verificação, que deve ser descrito pelo avaliador. Essa classificação não equivale a "atendido".
Não previsto nesta sprint	O item permanece no plano para uma sprint posterior e não recebe nota nesta entrega.

A rubrica é **cumulativa**: nas sprints 2 e 3, os itens aceitos anteriormente devem continuar funcionando. Uma regressão identificada no repositório deve ser apontada na avaliação atual.

Os critérios da rubrica completa totalizam 100 pontos. Em cada sprint, apenas os critérios previstos para ela compõem a pontuação máxima da entrega:

$$\text{Nota da sprint} = 10 \times \frac{\text{pontos obtidos nos critérios previstos e avaliáveis}}{\text{pontos máximos desses critérios}}$$

Um critério parcialmente atendido recebe **metade de seus pontos**. O resultado "não avaliável" exige justificativa e decisão do professor sobre a pontuação.

Exemplo de cálculo da nota da sprint:

Considere que a sua equipe optou pelas seguintes entregas na Sprint 1:

ES01 a ES09 (30 pts.), DW1 (3 pts.), DW2 (4 pts), BD01 (5 pts.) BD02 (5 pts.), DD01 (4 pts.) DD02 (4pts.)

então "pontos máximos desses critérios" é 55 pontos.

Considerando que os professores avaliaram os itens entregues e a equipe obteve 45 pontos, então o cálculo da nota da sprint 1 é dado por:

$$\text{Nota da sprint} = 10 \times \frac{45}{55} = 81{,}81$$

2. Engenharia de Software I — 30 pontos

Código	Critério	Pontos	O que verificar no GitHub
ES01	Product Backlog	4	Requisitos identificados e priorizados, cobrindo todo o desafio. Evidência: docs/backlog.md ou issues com RF, RNF e RP relacionados.
ES02	Planejamento da sprint	4	Objetivo, tarefas selecionadas, responsáveis e entregas definidos no início; alterações justificadas. Evidência: plano e registro da sprint.
ES03	Critérios de aceite	2	Condições observáveis para concluir as histórias ou tarefas. Evidência: critérios nas issues ou backlog, coerentes com os requisitos.
ES04	Rastreabilidade	3	Relacionar requisitos, tarefas, artefatos e código. Evidência: links entre issues, commits, PRs, SQL, protótipos e entregas.
ES05	Evolução incremental	2	Histórico de trabalho ao longo da sprint com mudanças compreensíveis e incrementos verificáveis. Não exigir quantidade mínima de commits.
ES06	Review e retrospectiva	3	Registrar entregas demonstradas, pendências, feedback e ao menos uma decisão de melhoria com acompanhamento na sprint seguinte.
ES07	Definition of Done	2	Definição de pronto objetiva e aplicada às entregas declaradas concluídas. Evidência: docs/dod.md e checklist no registro da sprint.
ES08	Documentação e execução	2	README e instruções compatíveis com a versão, equipe identificada, funcionalidades implementadas e links válidos. Execução reproduzível do incremento.
ES09	Participação da equipe	8	Contribuições relevantes e verificáveis de todos, em código, modelagem, Figma, conteúdo, testes, planejamento ou documentação, com evidências individuais.

Regra de ES09: 8 pontos se todos apresentarem contribuição relevante e verificável; 4 se houver participação limitada ou lacunas relevantes; 0 se ao menos um integrante não apresentar contribuição verificável. Pontuação coletiva. Commits isolados não medem participação. Para Figma, registrar autoria, tela ou componente e versão; dúvidas de autoria exigem análise do professor.

A qualidade conceitual das questões e dos materiais de estudo é avaliada em ES01 quando esses produtos estiverem previstos: cobertura dos temas, clareza, correção do gabarito, alternativas plausíveis e coerência entre estudo e avaliação. A aplicação técnica dessa base pertence a DW04; o tratamento visual pertence a DD03.

3. Desenvolvimento Web I — 30 pontos

Código	Critério	Pontos	O que verificar no GitHub
DW01	Tecnologias e organização	3	HTML, CSS e JavaScript efetivamente usados sem frameworks de frontend. Separar frontend, backend, banco e documentação; arquivos com nomes e responsabilidades compreensíveis. Avaliar em todas as sprints.
DW02	Interface e responsividade	4	HTML semântico, formulários com rótulos, layout responsivo em telas móveis e desktop; JavaScript para eventos, DOM e mensagens de validação. Avaliar implementação e uso das telas.
DW03	Backend e comunicação HTTP	5	Frontend comunica-se com backend funcional; endpoints com métodos, parâmetros, respostas JSON e códigos HTTP coerentes. Documentar API; tratar dados inválidos, falhas e operações de acesso ao PostgreSQL.
DW04	Estudo e apresentação das questões	5	Área de estudo com os 12 temas; banco final com 48 questões, quatro por tema, imagem e quatro alternativas com uma correta. Sortear uma questão por tema, registrar seleção e informar gabarito após encerramento.
DW05	Fluxo e regras da certificação	6	Cronômetro visível de 150 s; backend valida prazo, fechamento e resposta única. Prosseguir, interromper e retomar no próximo tema. Reconexão ou fechamento não reabre questão iniciada. Exibir progresso, histórico, nota e aprovação ≥65%; certificado com campos exigidos e QR para validação pública.
DW06	Autenticação e proteção dos dados	4	Cadastro com CPF único e login por CPF e senha; hash de senha, proteção das operações pessoais e autorização sobre registros. Nota e gabarito validados no backend.
DW07	Execução em Docker	3	Aplicação e PostgreSQL executáveis por Docker Compose, com Dockerfile(s), variáveis de exemplo e volume persistente. Demonstrar instalação a partir de clone limpo, inicialização e execução do incremento.

4. Modelagem de Banco de Dados— 25 pontos

Código	Critério	Pontos	O que verificar no GitHub
BD01	Modelo conceitual	5	DER com entidades, atributos, identificadores, relacionamentos e cardinalidades do domínio. Cobrir candidatos, temas, questões, alternativas, tentativas, respostas e certificados conforme o incremento; imagens e estudo podem ser atributos ou entidades, com justificativa.
BD02	Modelo lógico e normalização	5	Transformação coerente para relações, com PK, FK e restrições. Justificar normalização até a 3FN e eventuais exceções; evitar repetição de alternativas em colunas e duplicações sem necessidade.
BD03	Modelo físico e DDL	5	database/schema.sql com tabelas PostgreSQL, tipos, PK, FK, UNIQUE e NOT NULL. Executar em banco vazio e comparar com os modelos. CPF único e integridade referencial efetivos.
BD04	DML e consultas parametrizadas	5	SQL explícito para inserir, consultar e atualizar os dados do fluxo e para exclusões quando necessárias. Dados variáveis enviados como parâmetros separados ao driver. Evidência: SQL no backend e execução com dados conhecidos.
BD05	Persistência e integridade do histórico	5	Gravar e recuperar progresso, questão sorteada, resposta, resultado, horários e certificado. Impedir duplicação de respostas para o mesmo tema na mesma certificação. Registrar questão encerrada sem resposta e proteger a consistência do histórico.

O dicionário de dados deve informar tabela, coluna, tipo, significado, obrigatoriedade e restrições.

A cobertura das regras de certificação em execução é avaliada em DW04 e DW05; aqui se verifica a representação, a integridade e a persistência dos dados.

5. Design Digital — 15 pontos

Código	Critério	Pontos	O que verificar
DD01	Fluxos e wireframes	4	Mapear navegação e produzir wireframes das telas planejadas: início, cadastro, login, estudo, questão, feedback, progresso, histórico, resultado, certificado e validação pública. Incluir interrupção e retomada conforme a sprint.
DD02	Protótipo navegável no Figma	4	Protótipo de telas com interações que permitam percorrer o fluxo previsto. Incluir estados de erro, tempo esgotado e interrupção. Link com permissão de visualização e exportações da versão avaliada.

Código	Critério	Pontos	O que verificar
DD03	Consistência visual e legibilidade	4	Tipografia, cores, espaçamento, hierarquia, botões e componentes coerentes; rótulos claros e informação além da cor. Imagens das questões legíveis e relevantes para análise, com autoria ou fontes e adaptações identificadas.
DD04	Adaptação de telas e fidelidade	3	Projetar telas móveis e desktop e comparar com a interface implementada. Documentar ajustes justificados; realizar verificação de navegação com usuários ou colegas e registrar problemas e melhorias.

O Figma pode permanecer fora do GitHub. docs/design/figma.md deve reunir link, responsável, data, identificação da versão, telas e fluxo avaliado. Exportar as telas relevantes para docs/design/exportacoes/ e registrar as interações em texto ou vídeo. Um link para arquivo em constante edição, sem registro do estado entregue, não comprova a versão da sprint.

DD01 avalia a organização do fluxo; DD02, a navegabilidade do protótipo; DD03, a comunicação visual; DD04, a adaptação e a comparação entre projeto e implementação. DW02 avalia o comportamento técnico das telas no navegador.

6. Estrutura do repositório no GitHub

Utilizar um repositório para o projeto inteiro. A organização abaixo é um modelo; os nomes podem variar se os artefatos permanecerem fáceis de localizar. Criar pastas e documentos conforme a evolução, sem arquivos vazios ou anúncios de funcionalidades ainda não implementadas.

Caminho	Conteúdo esperado
README.md	Apresentação, integrantes, funcionalidades prontas, tecnologias, instalação, execução, sprints e links para documentos e Figma.
.gitignore e .env.example	Ignorar segredos e arquivos locais; indicar variáveis com valores fictícios e explicação.
compose.yaml	Serviços da aplicação e PostgreSQL, portas, variáveis e volume persistente.
frontend/	Dockerfile quando houver serviço próprio; index.html, pages/, css/, js/ e assets/. Scripts de DOM, requisições e apresentação das telas.
backend/	Dockerfile, configuração e código de rotas, regras da certificação e acesso parametrizado ao banco. Organização compatível com a tecnologia escolhida.
database/README.md	Comandos para criar banco e carregar dados, ordem dos scripts e localização do DML no backend.

database/schema.sql seed.sql	eDDL e dados fictícios para demonstração, incluindo temas, estudo, questões e alternativas. Imagens referenciadas com caminhos válidos.
docs/requisitos.md backlog.md	eRF, RNF e RP do desafio; 12 temas selecionados; histórias, prioridade, critérios de aceite e links para issues.
docs/plano-de-entregas.md dod.md	eCompromissos das três sprints e definição de pronto aplicada aos incrementos.
docs/arquitetura.md e api.md	Responsabilidades e fluxo frontend → backend → PostgreSQL; métodos, URLs, autenticação, parâmetros e respostas.
docs/database/	modelo-conceitual.png, modelo-logico.png, dicionario-de-dados.md e arquivos editáveis dos diagramas, quando disponíveis.
docs/design/	figma.md, fluxo-de-telas.md, exportacoes/ e registro de decisões e verificação da navegação.
docs/conteudo/	temas.md e questoes.md ou arquivos por tema: enunciado, imagem, alternativas, gabarito, justificativa e fontes; relação com seed.sql.
docs/sprints/sprint-N.md	Objetivo, commit, critérios previstos, concluídos e pendentes, verificações, Review, retrospectiva e contribuições individuais.
docs/evidencias/sprint-N/	Capturas, resultados e roteiro de verificação com dados fictícios. Evidências complementam código e artefatos.

Issues e quadro de acompanhamento podem substituir o backlog em Markdown, desde que o README forneça os links e cada entrega preserve os itens selecionados e sua situação no fechamento. Não exigir duas cópias manuais do mesmo backlog.

7. Modelo de documentação das entregas

Plano de entregas

Em docs/plano-de-entregas.md, informar objetivo de cada sprint e usar as colunas abaixo. Criar uma linha por critério ou incremento; incluir ES01 a ES09 e DW01 em todas as sprints. Para critérios parcialmente planejados, explicitar o que ficará para as próximas entregas.

Código e requisitos	Entrega e verificação	Responsáveis eSituação evidência
BD03 • RP02 e RP04	Criar cadastro e estruturaNome • link do SQL ePlanejado inicial de temas; executarmodelo schema.sql em PostgreSQL vazio e	

		conferir chaves e restrições.		
DD02 • RF03 e RF04	Percorrer cadastro e login no protótipo; conferir mensagens de validação.	Nome exportações	• Figma e evidência	Planejado
DW05 • RF10 a RF15	Encerrar questão aos 150s e retomar no próximo tema, inclusive após fechar navegador. Executar roteiro com candidato fictício.	Nome código	• e evidência	Planejado

Registro de cada sprint

docs/sprints/sprint-N.md deve conter: objetivo; prazo e horário de fechamento; tag sprint-N e hash do commit; critérios e requisitos planejados; entregas demonstradas; pendências; roteiro e resultados das verificações; links para evidências; feedback da Review; melhoria da retrospectiva; e contribuição de cada integrante.

No fechamento, criar uma tag sprint-1, sprint-2 ou sprint-3 e registrar o hash correspondente. A tag não deve ser movida após a entrega. Como issues e Figma podem mudar fora do histórico Git, preservar a situação dos itens e as exportações relevantes no commit entregue.

Roteiro do README principal

1. Nome do projeto e descrição do problema e da solução.
2. Equipe com nome, perfil no GitHub e responsabilidades na sprint.
3. Funcionalidades implementadas e pendentes, com links para o plano.
4. Tecnologias, organização das pastas e arquitetura.
5. Pré-requisitos, configuração das variáveis, criação e carga do banco e comando de execução com Docker Compose.
6. Endereços locais, candidato fictício para demonstração e roteiro básico de uso.
7. Links para requisitos, backlog, DoD, modelos de dados, API, Figma e conteúdo.
8. Tabela das sprints com período, objetivo, tag e registro da entrega.